Mohsin Iban Hossain

AIUB, Python Notes

PYTHON

Table of Contents

Sl. No	Topic	Pages
1	Pandas	03-50
2	NumPy	-
3	Matplotlib	-

INTRODUCTION TO PANDAS

Introduction

❖ What is Pandas?

❖ Pandas is a powerful **Python library** used for:

- Data analysis
- Data manipulation
- Working with structured data (tables)

➢ It helps you handle data like Excel, but using Python.

❖ Why Use Pandas?

❖ Pandas is widely used because:

- Easy to read and write data
- Handles large datasets efficiently
- Built-in tools for cleaning data
- Supports multiple file formats (CSV, Excel, JSON)
- Fast operations using optimized backend

❖ Real-Life Example?

❖ Imagine you have student data:

Name	Age	Marks
Alif	20	85
Jam	21	90
Nihan		87
Mohsin	22	

➢ With Pandas, you can:

- Filter students with marks $>$ 80
- Calculate average marks
- Remove missing values
- Sort data

Pandas vs NumPy

- Pandas is built on top of `NumPy`

Feature	Pandas	NumPy
Data Type	Table (DataFrame)	Array
Labels	Yes (rows & columns)	No
Use Case	Data analysis	Numerical computation

Installation

- Install Pandas using pip:

Command:

```
pip install pandas
```

Importing Pandas

Command:

```
import pandas as pd
```

Note: `pd` is just a shortcut (alias).

- **Basic Example:**

Example Code:

```
import pandas as pd

data = {
    "Name": ["Ali", "Rahim"],
    "Age": [20, 21]
}

df = pd.DataFrame(data)
print(df)
```

Output:

```
    Name  Age
0    Ali   20
1  Rahim   21
```

SERIES & DATAFRAME

Series

- A **Series** is a **1-dimensional** labeled array.
- Think of it like:
  - A single column in Excel
  - A list + index (labels)

Diagram of Series:

```
Index   Value
0       10
1       20
2       30
```

Creating a Series

Example code:

```
import pandas as pd

data = [10, 20, 30]
s = pd.Series(data)

print(s)
```

Output:

```
0    10
1    20
2    30
dtype: int64
```

Note: `dtype: int64`: This indicates the data type of the values in the Series. In this case, they are 64-bit integers.

Custom Index

Example code:

```
s = pd.Series([10, 20, 30], index=["a", "b", "c"])
print(s)
```

Output:

```
a    10
b    20
c    30
dtype: int64
```

Accessing Data

Example code:

```
print(s["a"])   # 10
```

Output:

```
10
```

DataFrame

⇨ A **DataFrame** is **2-dimensional** labeled data structure (rows + columns)

❖ **Diagram of Series:**

```
        Name    Age
Index
0       A       20
1       B       21
2       C       22
```

❖ **Visual Understanding:** Think like:

```
        Column →    Name      Age
Row ↓
        0           A         20
        1           B         21
```

Notes: DataFrame = **collection of Series**

Creating a Series

Example code:

```
data = {
    "Name": ["A", "B", "C"],
    "Age": [20, 21, 22]
}

df = pd.DataFrame(data)
df
```

Output:

	Name	Age
0	A	20
1	B	21
2	C	22

Accessing Columns

Example code:

```
df["Name"]
```

Output:

```
0    A
1    B
2    C
Name: Name, dtype: str
```

Note: Returns a **Series**

Accessing Rows

Example code:

```
data = {
    "Name": ["A", "B", "C"],
    "Age": [20, 21, 22]
}

df = pd.DataFrame(data)
print(df.iloc[0])    # first row
print(df.loc[2])    # by label
```

Output:

```
Name     A
Age     20
Name: 0, dtype: object

Name     C
Age     22
Name: 2, dtype: object
```

Notes:

- `df.iloc[0]` → Position-based indexing:
  - Returns the **first row**
  - It does **NOT care about the index label**
  - It only cares about **position (0 = first row, 1 = second row, etc.)**
- `df.loc[0]` → Label-based indexing
  - `loc` stands for **location**
  - It accesses rows/columns based on **index label (name)**
  - If no such label exists → **Error**

❖ **Example:**

Example code:

```
df = pd.DataFrame({
    "Name": ["A", "B", "C"],
    "Age": [20, 21, 22]
}, index=[10, 20, 30])

print(df.iloc[0])   # first row
print(df.loc[20])   # by label
```

Output:

```
Name     A
Age     20
Name: 10, dtype: object
Name     B
Age     21
Name: 20, dtype: object
```

Series vs DataFrame

Feature	Series	DataFrame
Dimension	1D	2D
Structure	Single column	Table
Example	[10,20,30]	Rows + Columns

Data Types in Pandas

- Pandas uses:
  - int64
  - float64
  - object (string)

Index Concept

- Every Series/DataFrame has an **index**:

Default index:	Custom index:
`0, 1, 2, 3...`	`df.index = ["a", "b", "c"]`

Shape of DataFrame

Example code:	Output:

`data = {`
`  "Name": ["A", "B", "C"],`
`  "Age": [20, 21, 22]`
`}`
 	
`df = pd.DataFrame(data)`
`df.shape`
	`(3, 2)`

Note: Returns 3 rows, 2 columns	

Important Attributes

Attribute	Meaning
`df.shape`	(rows, columns)
`len(df)`	number of rows
`df.columns`	column names
`df.index`	row labels

❖ **Example:**

Example code:

```
data = {
    "Name": ["A", "B", "C"],
    "Age": [20, 21, 22]
}

df = pd.DataFrame(data)
print(df.shape)
print(len(df))
print(df.columns)
print(df.index)
```

Output:

```
(3, 2)
3
Index(['Name', 'Age'], dtype='str')
RangeIndex(start=0, stop=3, step=1)
```

Practice Problem

Problem1: Create a Series of 5 numbers with custom index..

Code:

```
import pandas as pd
s = pd.Series([1,2,3,4,5], index=["a", "b", "c", "d", "e"])
print(s)
```

Output:

```
a    1
b    2
c    3
d    4
e    5
dtype: int64
```

Problem2: s = pd.Series([5,10,15], index=["x","y","z"]). Print value of "y"

Code:

```
print(s.loc["y"])
```

Output:

```
10
```

- **Problem3:** Create DataFrame.

Name	Marks
Ali	80
Sara	90

Code:

```
import pandas as pd
data = {
    "Name": ["Ali", "Sara"],
    "Marks": [80, 90]
}
df = pd.DataFrame(data)
df
```

Output:

	Name	Marks
0	Ali	80
1	Sara	90

- **Problem4:** From your DataFrame, Print only the **Marks column**

Code:

```
df["Marks"]
```

Output:

```
0    80
1    90
Name: Marks, dtype: int64
```

- **Problem5:** Print first row using iloc, Print first row using loc

Code:

```
print(df.iloc[0])
print(df.loc[0])
```

Output:

```
Name     Ali
Marks     80
Name: 0, dtype: object
Name     Ali
Marks     80
Name: 0, dtype: object
```

- **Problem6:** Create DataFrame. & Add a new column: Discount = Price - 10

Product	Price
A	100
B	200

Code:

```
import pandas as pd
data = {
    "Product":["A","B"],
    "Price":[100, 200]
}

df = pd.DataFrame(data)
df["Discounted"] = df["Price"] - 10
df
```

Output:

	Product	Price	Discounted
0	A	100	90
1	B	200	190

- **Problem7:** Create a DataFrame with the following data, and then add a new product **C** with price **150**.

Product	Price
A	100
B	200

Code:

```
import pandas as pd
data = {
    "Product":["A","B"],
    "Price":[100, 200]
}

df = pd.DataFrame(data)
df.loc[2] = ["C", 150]
print(df)
```

Output:

	Product	Price
0	A	100
1	B	200
3	C	150

DATA INPUT & OUTPUT (CSV, EXCEL, JSON)

Why Data Input/Output Matters

- In real life, data is stored in files like:
  - CSV
  - Excel
  - JSON
- Pandas helps you:
  - Load data into a DataFrame
  - Process it
  - Save results back to files

CSV Files

- CSV = **Comma-Separated Values**.

❖ **Example:**

data.csv

```
Name,Age,Marks
Ali,20,85
Sara,21,90
```

Reading CSV

Example code:

```
import pandas as pd

df = pd.read_csv("data.csv")
df
```

Output:

	Name	Age	Marks
0	Ali	20	85
1	Sara	21	90

Important Parameters

Parameter	Meaning
`header=0`	First row is column name
`index_col=0`	Use column as index
`usecols=["Name","Age"]`	Select columns

❖ **Example:**

Example code:	Output:

`df = pd.read_csv("data.csv",header=0, index_col=0, usecols=["Name", "Age"])`
 	
`print(df)`
	Age
	Name
	Ali 20
	Sara 21

Writing DataFrame to CSV

⇨ To save a DataFrame into a CSV file:

Syntax:

```
data = {
    "Name": ["Ali", "Sara"],
    "Age": [20, 21]
}
df = pd.DataFrame(data)
df.to_csv("output.csv", index=False)
```

Note:

1. By default, pandas also saves the **index column** (0,1,2,...)
2. `index=False` → **removes the index column** from the file

Useful Parameters

Parameter	Purpose
`index=False`	Remove index column
`columns=[...]`	Select specific columns
`sep=";"`	Change separator (default ,)
`header=False`	Remove column names
`mode='a'`	Append to file

❖ **Example:**

Example code:

```
import pandas as pd

# Create DataFrame
df = pd.DataFrame({
    "Product": ["A", "B", "C"],
    "Price": [100, 200, 150],
    "Stock": [10, 20, 15]
})

# Write to CSV with multiple options
df.to_csv(
    "output.csv",
    index=False,              # remove index column
    columns=["Product", "Price"],   # select only these columns
    sep=";",                  # use ; instead of ,
    header=True               # include column names
)

# Append more data (without header)
df.to_csv(
    "data.csv",
    mode='a',                 # append mode
    index=False,
    columns=["Product", "Price"],
    sep=";",
    header=False        # don't repeat column names
)
```

Output.csv

```
Product;Price
A;100
B;200
C;150
```

data.csv

```
A;100
B;200
C;150
```

Excel Files

Reading Excel

Example code:

```
df = pd.read_excel("students.xlsx")
df
```

Output:

	Name	Marks
0	Ali	85
1	Sara	90

Read specific sheet

Method 1:

```
df = pd.read_excel("data.xlsx", sheet_name="Sheet1")
```

Method 2:

```
df = pd.read_excel("data.xlsx", sheet_name=0)  # by index
```

Writing DataFrame to Excel File

⇨ To save a DataFrame into a Excel File:

Syntax:

```
df = pd.DataFrame({
    "Name": ["A", "B"],
    "Marks": [85, 90]
})

df.to_excel("students.xlsx", index=False)
```

Note: `index=False` → removes index column

Important Parameters

Parameter	Purpose
`sheet_name`	Specify sheet name
`index=False`	Remove index column
`usecols=[...]`	Read selected columns
`skiprows=n`	Skip first n rows
`nrows=n`	Read limited rows

JSON Files

⇨ JSON = **JavaScript Object Notation**.

❖ **Example:**

data.json

```
[
  {"Name": "Ali", "Age": 20},
  {"Name": "Sara", "Age": 21}
]
```

Reading JSON

Example code:

```
import pandas as pd

df = pd.read_json("data.json")
df
```

Output:

	Name	Age
0	Ali	20
1	Sara	21

Writing DataFrame to JSON File

Syntax:

```
df = pd.DataFrame({
    "Name": ["A", "B"],
    "Marks": [85, 90]
})

df.to_json("students.json", orient="records")
```

Common Parameters for orient

➪ Controls JSON structure:

Value	Format
`"records"`	List of dictionaries
`"columns"`	Default format
`"index"`	Indexed format

❖ **Example:**

Example code:

```
df = pd.DataFrame({
    "Name": ["A", "B"],
    "Marks": [85, 90]
})

df.to_json("students1.json",
orient="records",  indent=4)

df.to_json("students2.json",
orient="columns",  indent=4)

df.to_json("students3.json",
orient="index",  indent=4)
```

students1.json

```
[
    {
        "Name":"A",
        "Marks":85
    },
    {
        "Name":"B",
        "Marks":90
    }
]
```

students2.json

```
{
    "Name":{
        "0":"A",
        "1":"B"
    },
    "Marks":{
        "0":85,
        "1":90
    }
}
```

students2.json

```
{
    "0":{
        "Name":"A",
        "Marks":85
    },
    "1":{
        "Name":"B",
        "Marks":90
    }
}
```

Viewing Loaded Data

⇨ After loading a dataset (CSV, Excel, JSON), we usually want to **quickly inspect it**.

❖ View First Rows → `head()`

Example code:

```
data = {
    "Name": ["A", "B"],
    "Marks": [85, 90]
}

df = pd.DataFrame(data)
df.head()
```

Output:

	Name	Marks
0	A	85
1	B	90

Note: Shows **first 5 rows** (default)

```
data = {
    "Name": ["A", "B"],
    "Marks": [85, 90]
}

df = pd.DataFrame(data)
df.head(1)
```

	Name	Marks
0	A	85

Note: Shows first 1 rows (customs)

❖ View Last Rows → `tail()`

Example code:

```
data = {
    "Name": ["A", "B"],
    "Marks": [85, 90]
}

df = pd.DataFrame(data)
df.tail()
```

Output:

	Name	Marks
0	A	85
1	B	90

Note: Shows **last 5 rows** (default)

```
data = {
    "Name": ["A", "B"],
    "Marks": [85, 90]
}

df = pd.DataFrame(data)
df.last(1)
```

	Name	Marks
1	B	90

Note: Shows last 1 rows (customs)

❖ Quick Info → `info()`

Example code:

```
data = {
    "Name": ["A", "B"],
    "Marks": [85, 90]
}

df = pd.DataFrame(data)
df.info()
```

Output:

```
<class 'pandas.DataFrame'>
RangeIndex: 2 entries, 0 to 1
Data columns (total 2 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   Name    2 non-null      str
 1   Marks   2 non-null      int64
dtypes: int64(1), str(1)
memory usage: 164.0 bytes
```

Note: Shows:

- Column names
- Data types
- Non-null values
- Memory usage

❖ Statistical Summary → `describe()`

Example code:

```
data = {
    "Name": ["A", "B"],
    "Marks": [85, 90]
}

df = pd.DataFrame(data)
df.describe()
```

Output:

	Marks
count	2.000000
mean	87.500000
std	3.535534
min	85.000000
25%	86.250000
50%	87.500000
75%	88.750000
max	90.000000

CSV vs Excel vs JSON

Feature	CSV	Excel	JSON
Format	Text	Spreadsheet	Structured
Speed	Fast	Medium	Medium
Use	Most common	Business	APIs/Web

Practice Problem

- **Problem1:** Create a CSV file manually:

Name,Marks
Ali,80
Sara,90

Code:

```
import pandas as pd
data = {
    "Name": ["Ali", "Sara"],
    "Marks": [80, 90]
}

df = pd.DataFrame(data)
df.to_csv("data.csv", index=False)
```

- **Problem2:** Read a CSV file and print the first 5 rows.

Code:

```
df = pd.read_csv("data.csv")
print(df.head())
```

Problem3: Modify a column in a CSV file and save it as a new CSV.

Code:

```
df["Age"] = df["Age"] + 1
df.to_csv("data_modified.csv", index=False)
```

Problem4: Create a DataFrame and save it as an Excel file.

Code:

```
import pandas as pd
data = {
    "Name": ["Ali", "Sara"],
    "Marks": [80, 90]
}
df = pd.DataFrame(data)
df.to_excel("students.xlsx", index=False)
```

Problem5: Create a JSON file, load it, and convert it to CSV.

Code:

```
import pandas as pd
data = {
    "Name": ["Ali", "Sara"],
    "Marks": [80, 90]
}
df = pd.DataFrame(data)
df.to_json("data.json", orient="records", lines=True)
df = pd.read_json("data.json", lines=True)
df.to_csv("data.csv", index=False)
```

DATA INSPECTION & UNDERSTANDING DATA

Why Data Inspection is Important

⇨ Before doing anything (ML, analysis, etc.), you must:

- Understand structure
- Detect errors
- Identify missing values
- Know data types

❖ **Note: "Never trust data before inspecting it."**

A messy data set

messyData.csv

```
id,name,salary,city
1,Alice,65000,Dhaka
2,Bob,50000,Chittagong
3,Charlie,65000,Sylhet
4,Diana,,Dhaka
5,Ethan,65000,Khulna
6,Fiona,65000,Rajshahi
7,George,60000,Dhaka
8,Hannah,51000,Chittagong
9,Ian,50000,Sylhet
10,Jenny,34000,
11,Kevin,50000,Khulna
12,Linda,53000,Rajshahi
13,Michael,50000,Dhaka
14,Nina,34000,Chittagong
15,Oscar,55000,Sylhet
16,Paula,34000,Dhaka
17,Quentin,65000,Khulna
18,Rachel,34000,Rajshahi
19,Steve,65000,Dhaka
20,Tina,,Chittagong
20,Tina,,Chittagong
```

Viewing Data Quickly

❖ First 5 rows

Example code:

```
import pandas as pd
df = pd.read_csv("messyData.csv")
df.head()
```

Output:

	id	name	salary	city
0	1	Alice	65000.0	Dhaka
1	2	Bob	50000.0	Chittagong
2	3	Charlie	65000.0	Sylhet
3	4	Diana	NaN	Dhaka
4	5	Ethan	65000.0	Khulna

Note: Shows **first 5 rows** (default)

❖ First 3 rows

Example code:

```
import pandas as pd
df = pd.read_csv("messyData.csv")
df.head(3)
```

Output:

	id	name	salary	city
0	1	Alice	65000.0	Dhaka
1	2	Bob	50000.0	Chittagong
2	3	Charlie	65000.0	Sylhet

❖ last 5 rows

Example code:

```
import pandas as pd
df = pd.read_csv("messyData.csv")
df.head(3)
```

Output:

	id	name	salary	city
16	17	Quentin	65000.0	Khulna
17	18	Rachel	34500.0	Rajshahi
18	19	Steve	65000.0	Dhaka
19	20	Tina	NaN	Chittagong
20	20	Tina	NaN	Chittagong

❖ **Last 3 rows**

Example code:

```
import pandas as pd
df = pd.read_csv("messyData.csv")
df.head(3)
```

Output:

	id	name	salary	city
18	19	Steve	65000.0	Dhaka
19	20	Tina	NaN	Chittagong
20	20	Tina	NaN	Chittagong

Viewing Data Quickly

Example code:

```
df.info()
```

Output:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3 entries
Columns: 3 entries
Name      object
Age       int64
Marks     int64
```

Note: Gives:

- Number of rows
- Column names
- Data types
- Missing values

Statistical Summary

Example code:

```
df.describe()
```

Output:

	id	salary
count	21.000000	18.000000
mean	10.952381	47916.666667
std	6.127611	9558.257902
min	1.000000	34000.000000
25%	6.000000	39000.000000
50%	11.000000	48500.000000
75%	16.000000	53750.000000
max	20.000000	65000.000000

Column Names

Example code:	Output:
`df.columns`	`Index(['id', 'name', 'salary', 'city'], dtype='str')`

Index (Row Labels)

Example code:	Output:
`df.index`	`RangeIndex(start=0, stop=21, step=1)`

Shape of Data

Example code:	Output:
`df.shape`	`(21, 4)` (rows, columns)

Data Types

Example code:	Output:

`df.dtypes`	`id int64`
	`name str`
	`salary float64`
	`city str`
	`dtype: object`

Unique Values

Example code:	Output:

`df["name"].unique()`	`<StringArray>`
	`[ 'Alice', 'Bob', 'Charlie', 'Diana', 'Ethan', 'Fiona', 'George','Hannah', 'Ian', 'Jenny', 'Kevin', 'Linda', 'Michael', 'Nina', 'Oscar', 'Paula', 'Quentin', 'Rachel', 'Steve', 'Tina']`
	`Length: 20, dtype: str`

Note: Shows **distinct** values	

Count Unique Values

Example code:

```
df["name"].nunique()
```

Output:

```
20
```

Value Counts

Example code:

```
df["salary"].value_counts()
```

Output:

```
salary
65000.0    6
34000.0    5
50000.0    4
60000.0    1
51000.0    1
53000.0    1
55000.0    1
Name: count, dtype: int64
```

Checking Missing Data

Example code:

```
df.isnull().tail()
```

Output:

	id	name	salary	city
16	False	False	False	False
17	False	False	False	False
18	False	False	False	False
19	False	False	True	False
20	False	False	True	False

Count Missing Values

Example code:

```
df.isnull().sum()
```

Output:

```
id        0
name      0
salary    2
city      1
dtype: int64
```

Note: Column-wise missing count

Sample Data

Example code:

```
df.sample(2)
```

Output:

	id	name	salary	city
12	13	Michael	50000.0	Dhaka
20	20	Tina	NaN	Chittagong

Note: **Random** 2 rows

Memory Usage

Example code:

```
df.memory_usage()
```

Output:

```
Index     132
id        168
name      168
salary    168
city      168
dtype: int64
```

Note: Shows **memory used** by each column

Practice Problem

- **Problem1:** Load a dataset and perform the following operations:
  - Show the first 5 rows
  - Show the last 5 rows

Code:

```
df = pd.read_csv("messyData.csv")
df.head()
df.tail()
```

- **Problem2:** Find the number of missing values in each column.

Code:

```
df.isnull().sum()
```

- **Problem3:** Fill missing salary values with the mean salary.

Code:

```
mean_salary = df["salary"].mean()
df["salary"].fillna(mean_salary, inplace=True)
df.head()
```

- **Problem4:** Fill missing city values with "Unknown".

Code:

```
df["city"].fillna("Unknown", inplace=True)
```

- **Problem5:** Load a dataset and perform the following operations:
  - Check if there are duplicate rows
  - Remove duplicate rows from the dataset

Code:

```
#Check if there are duplicate rows
df  = pd.read_csv("messyData.csv")
duplicates = df.duplicated()
print(duplicates.sum())

#Remove duplicate rows from the dataset
df.drop_duplicates(inplace=True)
print(df.shape[0])
```

- **Problem6:** Load a dataset and perform the following operations:
  - Find the average salary.
  - Find the maximum and minimum salary
  - Count the number of employees in each city

Code:

```
#avg salary
avg = df["salary"].mean()
print(avg)

#max salary
max = df["salary"].max()
print(max)

#min salary
min = df["salary"].min()
print(min)

#Count the number of employees in each city
city_counts = df["city"].value_counts()
print(city_counts)
```

- **Problem7:** Load a dataset and perform the following operations:
  - Show all employees with salary greater than 50,000.
  - Create a new column `bonus` = 10% of salary.
  - Rename column `salary` to `monthly_salary`.

Code:

```
#Show all employees with salary greater than 50,000
salary_greater_than_50000 = df[df["salary"] > 50000]
print(salary_greater_than_50000)

#Create a new column bonus = 10% of salary.
df["bonus"] = df["salary"] * 0.1
print(df)

#Rename column salary to monthly_salary.
df = df.rename(columns={"salary": "monthly_salary"})
print(df)
```

DATA SELECTION & INDEXING

Why Selection & Indexing Matters

⇨ In real work, we never use the whole dataset—we:

- Pick specific columns
- Filter rows
- Extract meaningful data

A data set

data.csv

```
id,name,salary,city
1,Alice,65000,Dhaka
2,Bob,50000,Chittagong
3,Charlie,65000,Sylhet
4,Diana,52000,Dhaka
5,Ethan,65000,Khulna
6,Fiona,65000,Rajshahi
7,George,60000,Dhaka
8,Hannah,51000,Chittagong
9,Ian,50000,Sylhet
10,Jenny,34000,Dhaka
11,Kevin,50000,Khulna
12,Linda,53000,Rajshahi
13,Michael,50000,Dhaka
14,Nina,34000,Chittagong
15,Oscar,55000,Sylhet
16,Paula,34000,Dhaka
17,Quentin,65000,Khulna
18,Rachel,34000,Rajshahi
19,Steve,65000,Dhaka
20,Tina,48000,Chittagong
```

Selecting Columns

❖ **Single Column**

Example code:	Output:

`import pandas as pd`
`df = pd.read_csv("data.csv")`
`df["name"].head()`
	`0 Alice`
	`1 Bob`
	`2 Charlie`
	`3 Diana`
	`4 Ethan`
	`Name: name, dtype: str`

Note: Returns a **Series**	

❖ **Multiple Column**

Example code:	Output:
`df[["name", "salary"]].head()`	

	name	salary
0	Alice	65000
1	Bob	50000
2	Charlie	65000
3	Diana	52000
4	Ethan	65000

Note: Returns a **DataFrame**

Selecting Columns

❖ **Using `iloc` (Index-Based)**

Example code:	Output:

`df.iloc[0]`	`id 1`
	`name Alice`
	`salary 65000`
	`city Dhaka`
	`Name: 0, dtype: object`

Note: Returns **First row**	

Example code:	Output:
`df.iloc[0:2]`	

	id	name	salary	city
0	1	Alice	65000	Dhaka
1	2	Bob	50000	Chittagong

Note: Returns **First two** rows

❖ **Using loc (Label-Based)**

Example code:	Output:

`df.loc[0]`	`id 1`
	`name Alice`
	`salary 65000`
	`city Dhaka`
	`Name: 0, dtype: object`

Note: Returns Row with **label 0**

Example code:	Output:
`df.loc[0:1]`	

	id	name	salary	city
0	1	Alice	65000	Dhaka
1	2	Bob	50000	Chittagong

Note: Returns Rows from **label 0 to 1**

Difference

Method	Works On
`iloc`	Position (0,1,2)
`loc`	Label (index name)

Row + Column Selection

❖ **Using `iloc` (Index-Based)**

Example code:	Output:
`df.iloc[0, 1]`	`'Alice'`
Note: Returns **Row 0, Column 1 value**	

❖ **Using `loc` (Label-Based)**

Example code:	Output:
`df.loc[0, "name"]`	`'Alice'`
Note: Returns **Row 0, column "Name"**	

Selecting Block

❖ **Using `iloc` (Index-Based)**

Example code:	Output:
`df.iloc[0:2, 0:2]`	
Note: Returns **Rows 0–1, Columns 0–1**	

	id	name
0	1	Alice
1	2	Bob

Filtering Data

❖ **Basic Filtering**

Example code:	Output:
`df[df["salary"] > 50000].head(2)`	

	id	name	salary	city
0	1	Alice	65000	Dhaka
2	3	Charlie	65000	Sylhet

❖ Multiple Conditions

Example code:

```
df[(df["id"] > 15) & (df["salary"] > 50000)]
```

Output:

	id	name	salary	city
16	17	Quentin	65000	Khulna
18	19	Steve	65000	Dhaka

❖ OR Condition

Example code:

```
df[(df["id"] > 15) | (df["salary"] > 50000)].sample(5)
```

Output:

	id	name	salary	city
4	5	Ethan	65000	Khulna
15	16	Paula	34000	Dhaka
3	4	Diana	52000	Dhaka
5	6	Fiona	65000	Rajshahi
0	1	Alice	65000	Dhaka

Selecting Specific Columns After Filter

Example code:

```
df[df["id"] > 15][["name", "salary"]]
```

Output:

	name	salary
15	Paula	34000
16	Quentin	65000
17	Rachel	34000
18	Steve	65000
19	Tina	48000

Setting Conditions with `loc`

Example code:	Output:
`df.loc[df["id"] > 15, "name"]`	15 Paula 16 Quentin 17 Rachel 18 Steve 19 Tina Name: name, dtype: str
Note: Only names **where condition is true**	

Boolean Indexing

Example code:	Output:
`df["id"] > 15`	13 False 14 False 15 True 16 True 17 True 18 True 19 True Name: id, dtype: bool
Note: This **boolean array is** used for filtering	

Index-Based Selection Tricks

❖ **Negative Index**

Example code:	Output:
`df.iloc[-1]`	id 20 name Tina salary 48000 city Chittagong Name: 19, dtype: object
Note: Return **Last row**	

❖ **Step Selection**

Example code:

```
df.iloc[::2].head(3)
```

Output:

	id	name	salary	city
0	1	Alice	65000	Dhaka
2	3	Charlie	65000	Sylhet
4	5	Ethan	65000	Khulna

Note: Return **Every 2nd row**

Setting Values

Example code:

```
df.loc[0, "name"] = "John Doe"
df.loc[0, "salary"] = 60000
print(df.loc[0, ["name", "salary"]])
```

Output:

```
name      John Doe
salary       60000
Name: 0, dtype: object
```

Note: Modify data

Dropping Rows/Columns

Example code:

```
df.drop("Age", axis=1)    # column
print(df.head(2))

df.drop(0, axis=0)        # row
print(df.head(2))
```

Output:

```
   id   name  salary        city
0   1  Alice   65000       Dhaka
1   2    Bob   50000  Chittagong

   id   name  salary        city
0   1  Alice   65000       Dhaka
1   2    Bob   50000  Chittagong
```

DATA CLEANING

What is Data Cleaning?

⇨ Data cleaning means: **Fixing messy, incomplete, or incorrect data**

❖ **Real Problems in Data:**

- Missing values (NaN)
- Duplicate rows
- Wrong data types
- Inconsistent formats

A data set

messyData.csv

```
id,name,salary,city
1,Alice, ,
,Bob,,Chittagong
3,Charlie, ,
4,Diana,52000,Dhaka
5,Ethan,65000,Khulna
6,Fiona, ,Rajshahi
7, ,60000,Dhaka
8,Hannah,51000,Chittagong
9,Ian,50000,Sylhet
10,Jenny,34000,
11, ,50000,Khulna
12,Linda,53000,Rajshahi
13,Michael,50000,Dhaka
14,Nina,34000,
15, ,55000,Sylhet
16,Paula,34000,Dhaka
17, ,65000,Khulna
18,Rachel,34000,
19, ,65000,Dhaka
20,Tina,48000,
```

Handling Missing Values

❖ **Detect Missing Values:**

Example code:

```
df.isnull().head()
```

Output:

	id	name	salary	city
0	False	False	False	True
1	True	False	True	False
2	False	False	False	True
3	False	False	False	False
4	False	False	False	False

❖ **Count Missing Values:**

Example code:

```
df.isnull().sum()
```

Output:

```
id        1
name      0
salary    1
city      6
dtype: int64
```

Note: **Column-wise** missing count

❖ **Drop Missing Values:**

Example code:

```
df.dropna().head()
```

Output:

	id	name	salary	city
3	4.0	Diana	52000	Dhaka
4	5.0	Ethan	65000	Khulna
5	6.0	Fiona		Rajshahi
6	7.0		60000	Dhaka
7	8.0	Hannah	51000	Chittagong

Note: **Removes rows** with missing values

❖ **Drop specific column missing:**

Example code:

```
df.dropna(subset=["name"]).head(2)
```

Output:

	id	name	salary	city
3	4.0	Diana	52000	Dhaka
4	5.0	Ethan	65000	Khulna

Note: **Removes rows** with missing values

❖ **Fill Missing Values:**

Example code:

```
df.fillna(0).head()
```

Output:

	id	name	salary	city
0	1.0	Alice		0
1	0.0	Bob	0	Chittagong
2	3.0	Charlie		0
3	4.0	Diana	52000	Dhaka
4	5.0	Ethan	65000	Khulna

Note: Replace **NaN** with 0

Handling Duplicates

❖ **Detect duplicates:**

Example code:

```
print(df.duplicated().sum())
```

Output:

```
0
```

❖ **Remove duplicates:**

Example code:

```
df.drop_duplicates().shape[0]
```

Output:

```
20
```

❖ **Remove based on column:**

Example code:	Output:
`df.drop_duplicates(subset=["name"]).shape[0]`	16

Renaming Columns

Example code:	Output:
`df.rename(columns={"name": "Student_Name"}, inplace=True)` `df.columns`	`Index(['id', 'Student_Name', 'salary', 'city'], dtype='str')`

Changing Data Types

❖ **Check types:**

Example code:	Output:

`df.dtypes`	`id float64`
	`Student_Name str`
	`salary str`
	`city str`
	`dtype: object`

❖ **Convert type:**

Example code:	Output:
`df["id"] = df["id"].astype(str)` `print(df["id"].dtype)`	`str`

Handling Outliers

Example code:	Output:
`df = pd.read_csv("example.csv")` `df[df["Marks"] > 100]`	

	Name	Age	Marks
1	Sara	21	190

DATA MANIPULATION

Data Manipulation

⇨ Data manipulation means: **Changing, transforming, and preparing data**

❖ **Why it matters:**

- Feature engineering for ML
- Data transformation
- Custom calculations

A data set

data.csv

```
id,name,salary,city
1,Alice,65000,Dhaka
2,Bob,50000,Chittagong
3,Charlie,5000,Sylhet
4,Diana,52000,Dhaka
5,Ethan,65000,Khulna
```

Adding New Columns

Example code:

```
df = pd.read_csv("data.csv")
df["age"] = [23, 24, 25, 26, 27]
df
```

Output:

	id	name	salary	city	age
0	1	Alice	65000	Dhaka	23
1	2	Bob	50000	Chittagong	24
2	3	Charlie	5000	Sylhet	25
3	4	Diana	52000	Dhaka	26
4	5	Ethan	65000	Khulna	27

❖ **New column based on existing data:**

Example code:

```
df["Bonus"] = df["salary"] * 0.1
df
```

Output:

	id	name	salary	city	age	Bonus
0	1	Alice	65000	Dhaka	23	6500.0
1	2	Bob	50000	Chittagong	24	5000.0
2	3	Charlie	5000	Sylhet	25	500.0
3	4	Diana	52000	Dhaka	26	5200.0
4	5	Ethan	65000	Khulna	27	6500.0

Sorting Data

❖ **Sort by column (ascending order):**

Example code:

```
df.sort_values(by="Bonus")
```

Output:

	id	name	salary	city	age	Bonus
2	3	Charlie	5000	Sylhet	25	500.0
1	2	Bob	50000	Chittagong	24	5000.0
3	4	Diana	52000	Dhaka	26	5200.0
0	1	Alice	65000	Dhaka	23	6500.0
4	5	Ethan	65000	Khulna	27	6500.0

❖ **Sort by column (Descending order):**

Example code:

```
df.sort_values(by="Bonus", ascending=False)
```

Output:

	id	name	salary	city	age	Bonus
0	1	Alice	65000	Dhaka	23	6500.0
4	5	Ethan	65000	Khulna	27	6500.0
3	4	Diana	52000	Dhaka	26	5200.0
1	2	Bob	50000	Chittagong	24	5000.0
2	3	Charlie	5000	Sylhet	25	500.0